

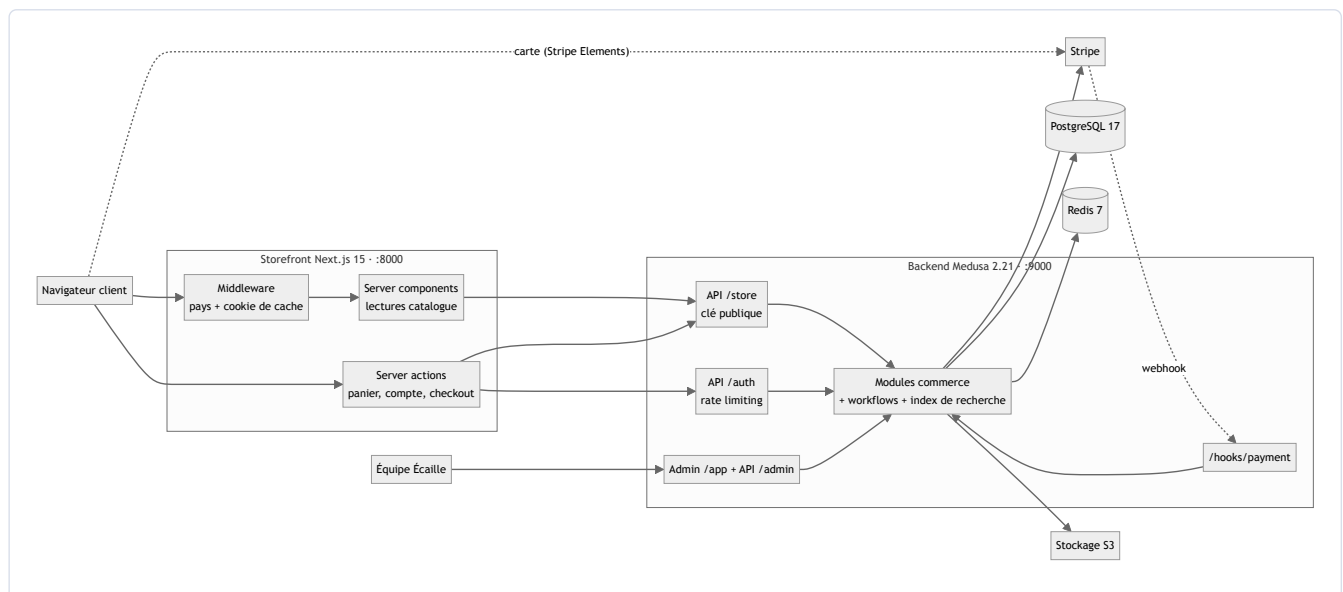
L'ARCHITECTURE

Un storefront qui rend tout côté serveur, un backend Medusa découpé en modules isolés, et une infrastructure activée par l'environnement. Comment les pièces se parlent, et où passe chaque requête.

Dépôt [grintzdel/medusa](#) État de `main` au 8 octobre 2026

SYSTÈME

QUI PARLE À QUI



LE NAVIGATEUR NE VOIT JAMAIS LE BACKEND

Les appels à Medusa partent du serveur Next (`MEDUSA_BACKEND_URL` n'est pas public). Seuls Stripe et les images sortent directement du navigateur.

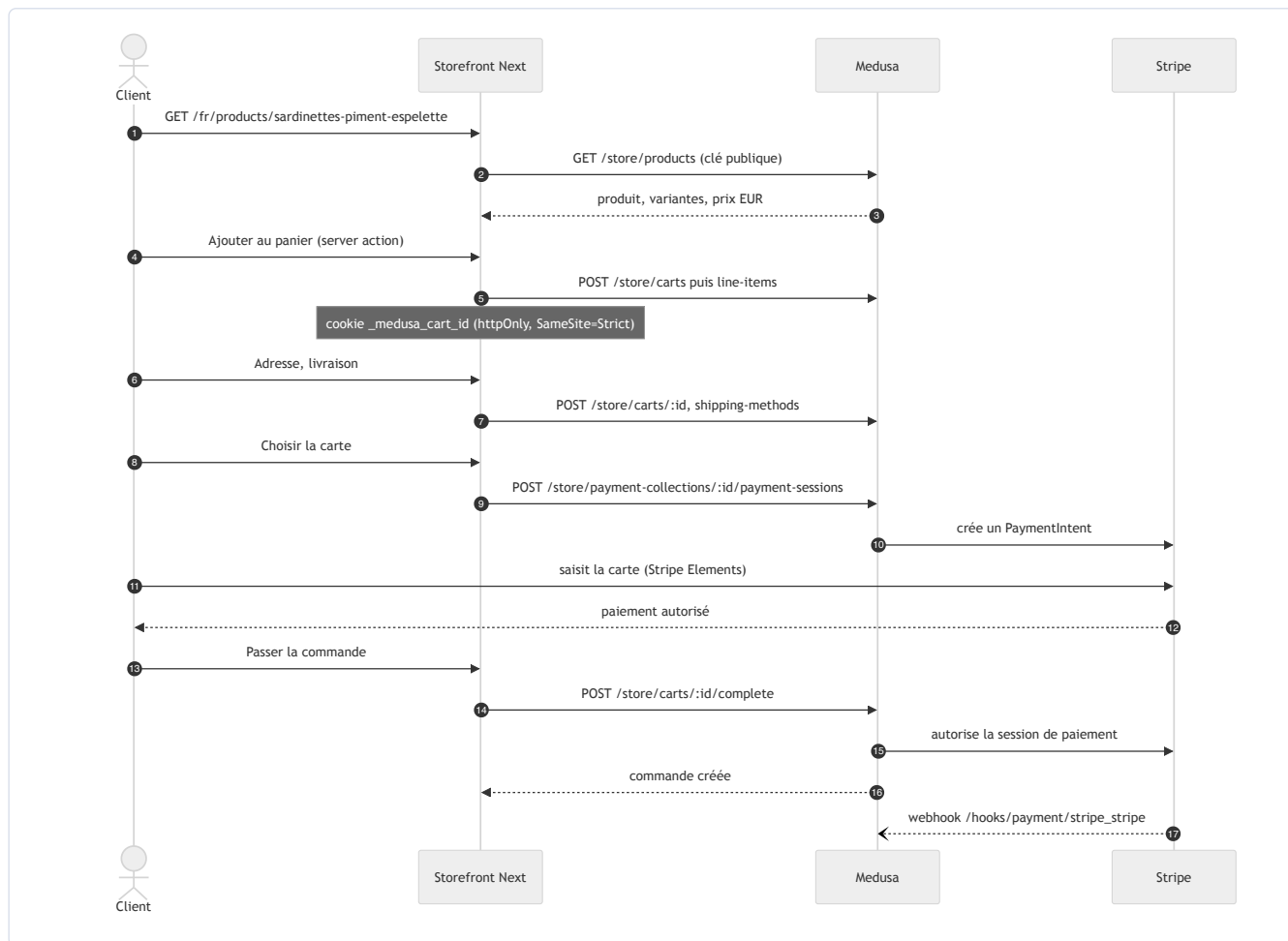
LES MODULES NE SE CONNAISSENT PAS

Chaque module Medusa a ses tables ; les relations passent par des tables de liens. Voir le modèle de données.

LA LOGIQUE MÉTIER EST DANS DES WORKFLOWS

Les routes résolvent un workflow et le lancent. Un workflow compose des étapes avec compensation en cas d'échec.

LE CHEMIN D'UN ACHAT



Le paiement reste en `requires_capture` : l'équipe capture depuis l'admin, comme le veut le réglage par défaut de Medusa. Capture manuelle ou automatique : décision ouverte dans #38.

POINTS D'ENTRÉE

CE QUE LE CODE AJOUTE À MEDUSA

OÙ	QUOI	POURQUOI
BACKEND		
<code>medusa-config.ts</code>	Modules Redis, Stripe et S3 selon l'environnement ; refus de démarrer en prod si une variable manque	ADR 0003, #21, #27
<code>src/api/middlewares.ts</code> <code>src/api/utils/rate-limit.ts</code>	Limitation de débit sur <code>/auth</code> ; <code>/store/search</code> restreint aux produits publiés du canal de vente	ADR 0004, #26
<code>src/search/</code>	Index produit : titre, description, catégories, options, prix min/max par devise ; réindexé sur les événements produit	#34, #51
<code>src/scripts/</code>	Seed du catalogue Écaille (idempotent), nettoyage de la démo Medusa, seed de démo pour la pagination	#19, #46
STOREFRONT		
<code>src/middleware.ts</code>	Pays depuis l'URL, sinon <code>x-vercel-ip-country</code> , sinon <code>NEXT_PUBLIC_DEFAULT_REGION</code> ; cookie <code>_medusa_cache_id</code> posé sans redirection	#30

OÙ	QUOI	POURQUOI
<code>src/lib/data/</code>	Server actions et lectures vers l'API (panier, client, commandes, paiement...)	Un seul endroit qui parle au backend
<code>src/lib/config.ts</code>	Client du SDK Medusa ; transmet l'IP du visiteur sur <code>/auth</code> uniquement	Sinon un client bloque toute la boutique (#26)
<code>next.config.js</code>	CSP, HSTS, <code>X-Frame-Options</code> ... ; domaines d'images autorisés	ADR 0006, #25
<code>src/modules/</code>	Composants par domaine : produits, panier, checkout, compte, commande	Structure du starter Medusa

DÉPÔT

ARBORESCENCE

```
.
├── apps/
│   ├── backend/                # Medusa (@dtc/backend)
│   │   ├── medusa-config.ts    # modules activés par l'environnement
│   │   └── src/
│   │       ├── api/            # middlewares, rate limiting
│   │       ├── search/         # index produit de /store/search
│   │       ├── scripts/        # seeds
│   │       ├── migration-scripts/
│   │       └── __tests__/       # tests Jest (hors de src/search)
│   └── storefront/            # Next.js (@dtc/storefront)
│       ├── e2e/                # Playwright
│       └── src/
│           ├── app/[countryCode]/(main|checkout)/
│           ├── lib/data/        # accès à l'API
│           ├── modules/         # composants par domaine
│           └── middleware.ts
├── docs/adr/                   # décisions d'architecture
├── scripts/e2e.sh               # E2E sur base jetable
├── docker-compose.yml           # Postgres 17 + Redis 7 sur 127.0.0.1
├── turbo.json
└── .github/workflows/ci.yml     # checks, audit, e2e
```

ENVIRONNEMENTS

OÙ ÇA TOURNE

ENVIRONNEMENT	STOREFRONT	BACKEND	BASE	REDIS	STATUT
Dev local	:8000	:9000	medusa	db 0	EN PLACE
E2E local (<code>pnpm test:e2e</code>)	:8001	:9001	medusa_e2e	db 1	EN PLACE
CI GitHub Actions	:8001	:9001	conteneur	conteneur	EN PLACE
Production	—	derrière un seul reverse proxy	managée	managé	À FAIRE · #42

LES PROTECTIONS, COUCHE PAR COUCHE

NAVIGATEUR

CSP (Stripe autorisé, `frame-ancestors 'none'`), HSTS, `X-Frame-Options: DENY`, `Referrer-Policy`, `Permissions-Policy`.

SESSION

Cookies `_medusa_jwt` et panier en `httpOnly`, `SameSite=Strict`, `Secure` en prod.

API

10 connexions / 15 min, 10 inscriptions / h, 5 réinitialisations / h par IP. Au-delà : 429.

CONFIGURATION

Démarrage refusé en prod avec des secrets de dev, sans CORS ni Redis, ou Stripe sans secret de webhook.

DÉPENDANCES

`pnpm audit` bloquant sur le critique, Dependabot, correctifs transitifs par `pnpm.overrides`.

LIMITES CONNUES

`'unsafe-inline'` sur les scripts (#40) ; IP forgeable sans reverse proxy ; 2 failles « high » sans correctif (#41).